

Reinforcement Learning

CS 59300: RL1

October 28, 2025

Joseph Campbell
Department of Computer Science

Project proposals due Oct 30

2-page research-style paper outlining your project and progress

IEEE-style format

Template:

<https://ras.papercept.net/conferences/support/support.php>

Midterm November 4

Will release “high-level” study guide
In class at regular time and location

Topics:

- Bandits/MDPs
- Bellman equations, policy/value iteration
- DQNs
- Actor-Critic
- Multi-Agent RL
- Exploration

Today's lecture

1. Inverse reinforcement learning continued
2. Generative adversarial imitation learning
3. Offline reinforcement learning
4. Imitation learning + reinforcement learning

Some content inspired by Sergey Levine's Berkeley CS285

Inverse reinforcement learning cont.

Recap: Inverse reinforcement learning

Given a set of demonstrations, instead of inferring policy (BC) instead infer reward function

With reward function, perform standard reinforcement learning to learn policy

This is another type of **imitation learning**

Goal: learn teacher's reward function

Learn reward function for teacher policy π^* such that

$$\pi^* = \arg \max_{\pi \in \Pi} \mathbb{E}[r^*(s, a)]$$

Idea: given (state, action, next state) triplets, try to estimate what reward the teacher may be maximizing when choosing actions

- Literally *inverse* reinforcement learning
- Instead of learning policy from reward, learn reward from policy

Maximum entropy IRL

Remember that IRL is ill-posed

- Many reward functions can correspond to the same policy
- Many mixtures of policies correspond to same feature expectation

Instead of arbitrary selecting a policy, e.g. max Q margin...

...use the **principle of maximum entropy!**

Principle of maximum entropy

The probability distribution which best represents the current state of knowledge about a system is the one with the largest entropy.

- E.T. Jaynes, 1957

Occam's Razor: *The simpler explanation is to be preferred.*

- The distribution with the largest entropy (uncertainty) makes the **least assumptions** about the true distribution

Example

Suppose in a robot navigation task the “expert” always drives near the northwest corner of the room.

Is it because...

- They prefer the northwest corner?
- They dislike the southeast corner?
- They just like moving in that direction?

MaxEnt IRL tries to resolve this ambiguity by assuming that the expert is **not deterministically optimal**, but instead **favors high reward paths** while behaving **stochastically**.

MaxEnt IRL with linear reward formulation

$$p(\tau|\theta) = \frac{1}{Z(\theta)} e^{\sum_{s_t \in \tau} \theta^T \phi(s_t)}$$

Normalization constant
(softmax dist. over trajectories)

Exponential sum of rewards = trajectories with
higher rewards are exponentially more likely

$$\theta^* = \arg \max_{\theta} L(\theta) = \arg \max_{\theta} \sum_{\tau_i \in \mathcal{D}} \log p(\tau|\theta)$$

MaxEnt IRL with linear reward formulation

Idea: make the state-visitation frequency under the learned policy match the state-visitation frequency of the expert.

Starting with randomly initialized reward function...

1. Compute optimal policy for current reward
2. Estimate expected state visitation frequency
3. Adjust reward function to match expert's state visitation frequency

Repeat until convergence

Generative adversarial imitation learning

What is the problem with IRL?

Many algorithms are iterative...

Starting with randomly initialized reward function...

1. Compute optimal policy for current reward
2. Estimate expected state visitation frequency
3. Adjust reward function to match expert's state visitation frequency

Repeat until convergence

What is the problem with IRL?

Many algorithms are iterative...

Starting with randomly initialized reward function...

1. Compute optimal policy for current reward
2. Estimate expected state visitation frequency
3. Adjust reward function to match expert's state visitation frequency

Repeat until convergence

”Compute optimal policy” is RL in an inner loop!

- Very computationally expensive...

Generative models

Generative model: learns a joint distribution p_θ to “match” data p_{data}

$$p_\theta(x, y)$$

Discriminative model: learns a conditional distribution

$$p_\theta(y|x)$$

Generative adversarial network

Idea: A **generator** G learns the probability distribution of the data and generates synthetic outputs, and a **discriminator** D estimates the probability that the samples are real or synthetic.

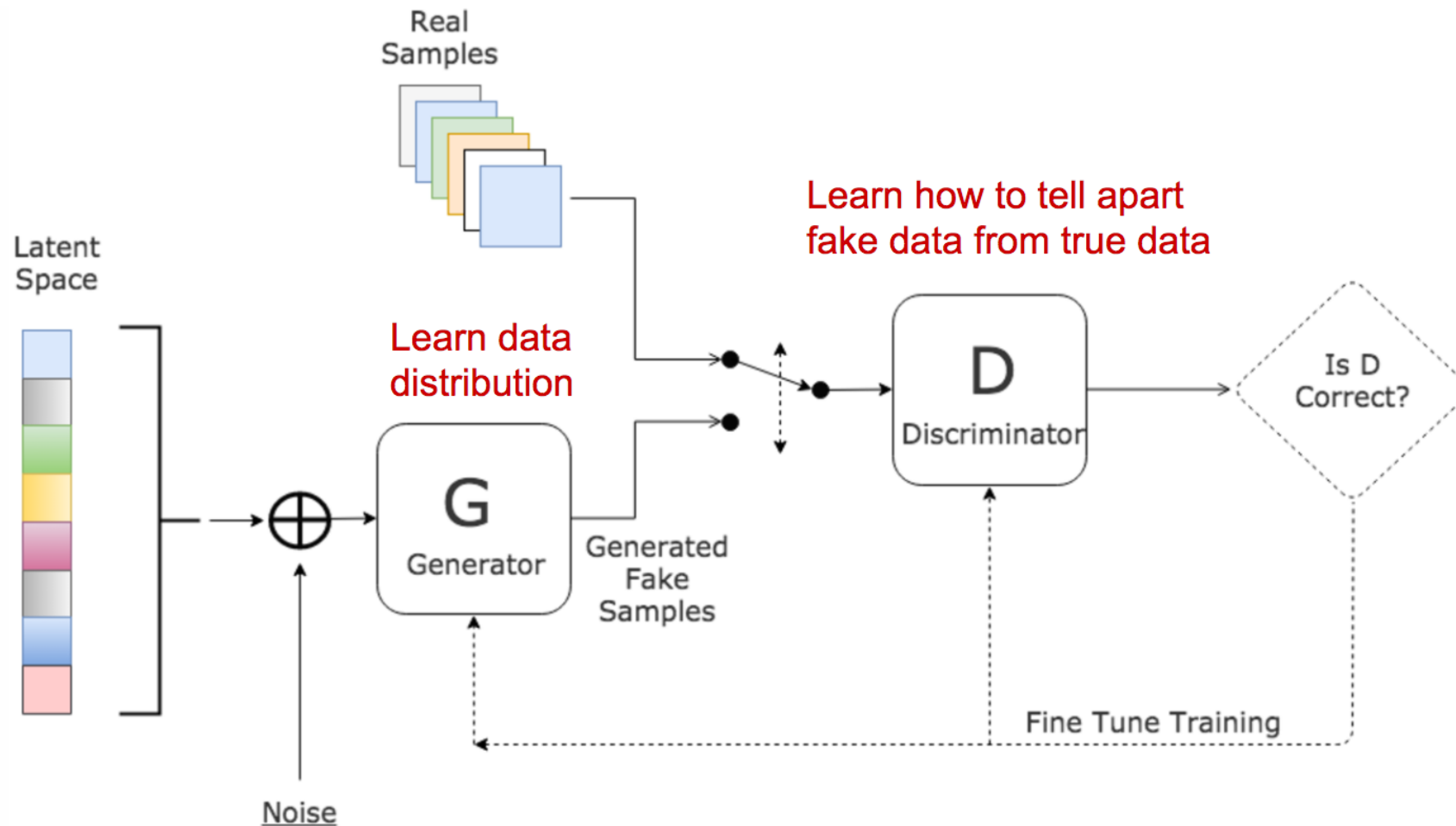
This results in a **competition!**

Generator: tries to fool the discriminator

Discriminator: tries not to be fooled

Generative adversarial network

$$\min \max V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log 1 - D(G(z))]$$



Generative adversarial network

$$\min \max V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log 1 - D(G(z))]$$

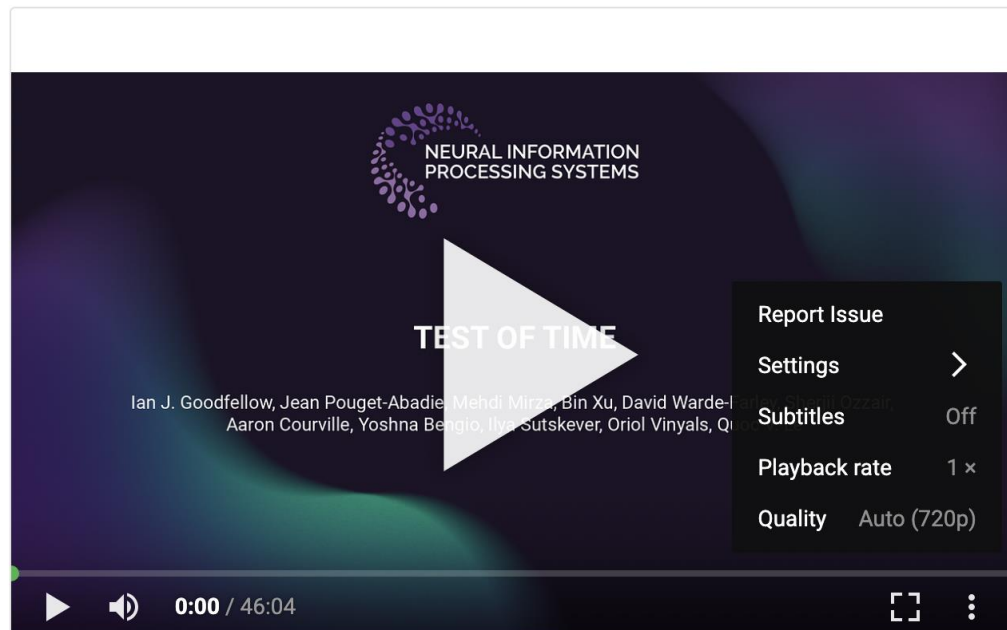
Discriminator wants to maximize
accuracy over *real data*

Generator wants to minimize
discriminator's accuracy over *fake data*

This is a zero-sum minimax game!

Generative adversarial networks

Recently won test-of-time award at
NeurIPS 2024



<https://neurips.cc/virtual/2024/test-of-time/105032>

Generative Adversarial Nets

Ian J. Goodfellow,^{*} Jean Pouget-Abadie,^{*} Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair,[†] Aaron Courville, Yoshua Bengio[‡]
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

1 Introduction

The promise of deep learning is to discover rich, hierarchical models [2] that represent probability distributions over the kinds of data encountered in artificial intelligence applications, such as natural images, audio waveforms containing speech, and symbols in natural language corpora. So far, the most striking successes in deep learning have involved discriminative models, usually those that map a high-dimensional, rich sensory input to a class label [14, 22]. These striking successes have primarily been based on the backpropagation and dropout algorithms, using piecewise linear units [19, 9, 10] which have a particularly well-behaved gradient. Deep *generative* models have had less of an impact, due to the difficulty of approximating many intractable probabilistic computations that arise in maximum likelihood estimation and related strategies, and due to difficulty of leveraging the benefits of piecewise linear units in the generative context. We propose a new generative model estimation procedure that sidesteps these difficulties.¹

Generative adversarial imitation learning

Idea: combine GANs with imitation learning

Let the state $x = (s, a)$

Generator: student policy which seeks to generate actions that look like the teacher's

Discriminator: tries to identify which actions were taken by the student and which by the teacher

Generative adversarial imitation learning

1. Sample trajectory from student
2. Update discriminator, which tries to classify student from teacher
3. Update student, which tries to minimize discriminator's accuracy

Algorithm 1 Generative adversarial imitation learning

- 1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
- 2: **for** $i = 0, 1, 2, \dots$ **do**
- 3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
- 4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

$$\hat{\mathbb{E}}_{\tau_i} [\nabla_w \log(D_w(s, a))] + \hat{\mathbb{E}}_{\tau_E} [\nabla_w \log(1 - D_w(s, a))] \quad (17)$$

- 5: Take a policy step from θ_i to θ_{i+1} , using the TRPO rule with cost function $\log(D_{w_{i+1}}(s, a))$. Specifically, take a KL-constrained natural gradient step with

$$\begin{aligned} & \hat{\mathbb{E}}_{\tau_i} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q(s, a)] - \lambda \nabla_{\theta} H(\pi_{\theta}), \\ & \text{where } Q(\bar{s}, \bar{a}) = \hat{\mathbb{E}}_{\tau_i} [\log(D_{w_{i+1}}(s, a)) \mid s_0 = \bar{s}, a_0 = \bar{a}] \end{aligned} \quad (18)$$

- 6: **end for**
-

Doesn't require an inner RL loop

GAIL does not require an inner RL loop, making it more computationally feasible than IRL

- Directly learn a policy rather than a cost function

Unlike BC, tries to match the **distribution** of the teacher, not simply learn a point-wise mapping from states to actions

- Also makes it better for multi-modal policies (next lecture)

Offline reinforcement learning

What's the problem with imitation learning?

Consider a **sub-optimal teacher**. How good is the resulting student?

What's the problem with imitation learning?

Consider a **sub-optimal teacher**. How good is the resulting student?

The student will never* exceed the teacher's performance!

- **Unless we augment the demonstrations...*

So how can we overcome this limitation and learn from even sub-optimal demonstrations?

Offline reinforcement learning

Use teacher demonstrations to learn how to act

- Same as imitation learning

Key difference:

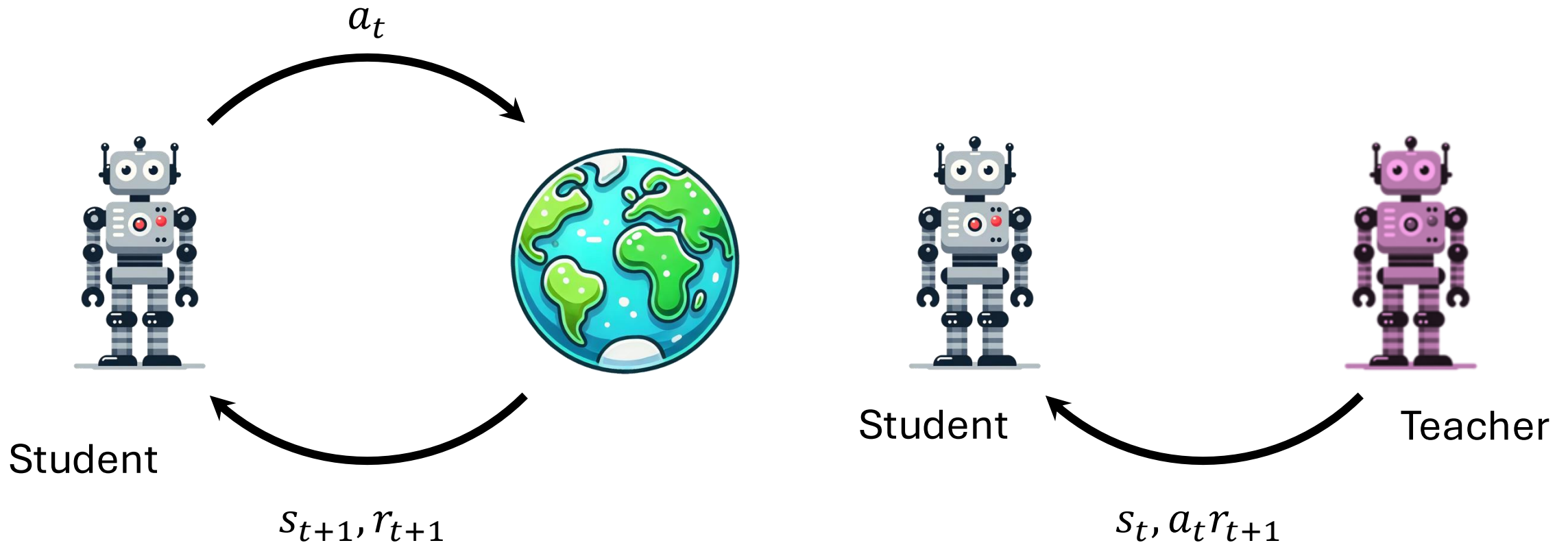
Instead of state-action pairs we have **state-action-reward triplets**

Reward serves as additional piece of information to help guide imitation learning

- Low-reward actions: ignore
- High-reward actions: follow

Offline reinforcement learning

The student tries to learn and **imitate** the teacher's policy



Isn't this just off-policy learning?

Yes and no!

Yes:

- We use previously collected experience just like in off-policy

No:

- The agent is not allowed to interact with the environment to collect more

Can we still just apply off-policy algorithms?

Hint: think about the i.i.d. assumption violation of behavior cloning

Can we still just apply off-policy algorithms?

Hint: think about the i.i.d. assumption violation of behavior cloning

Yes, you can. But it doesn't work well!

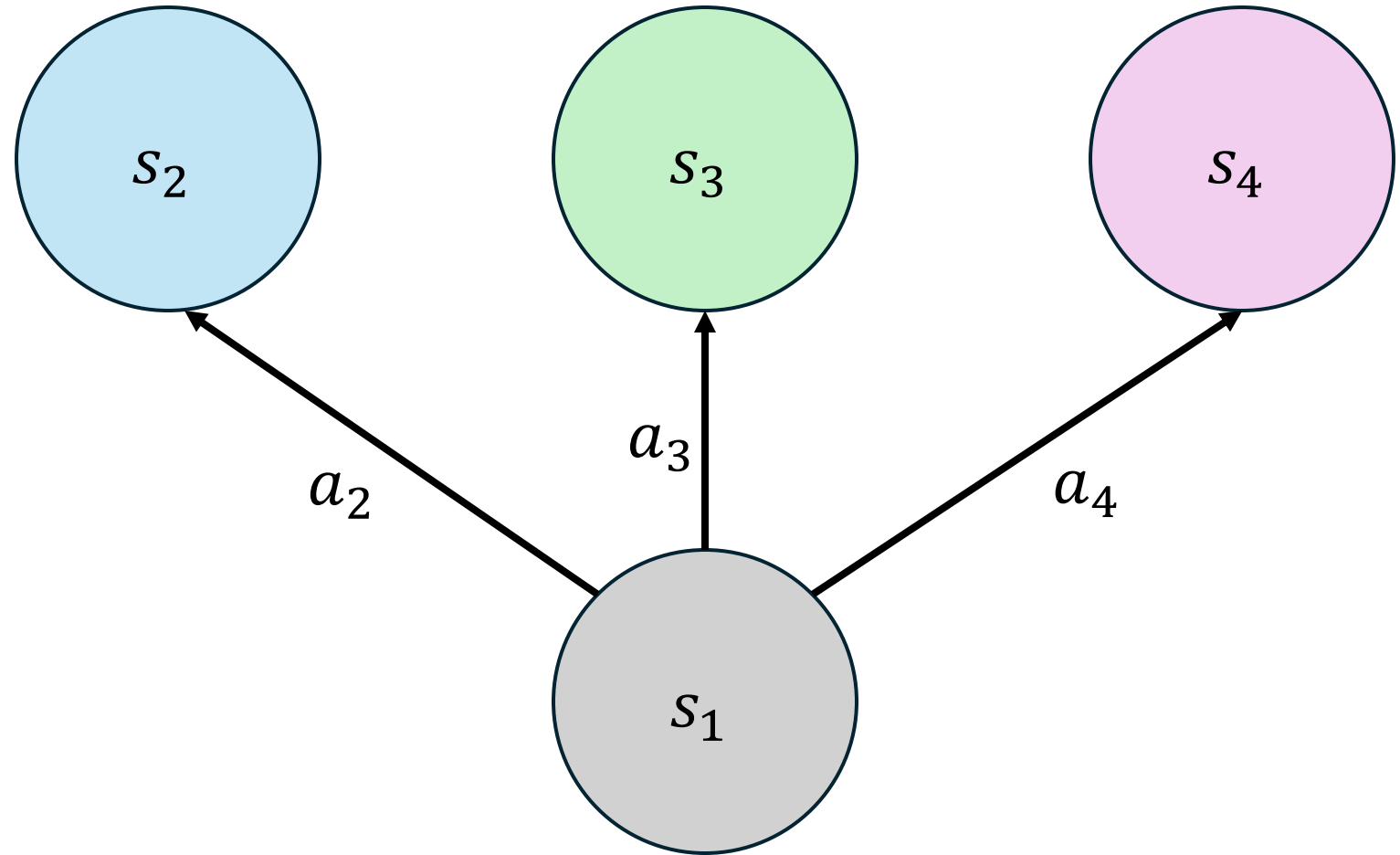
Extrapolation error:

- Estimating the state/action-value for unseen state-action pairs

Extrapolation error

We want to estimate...

- $Q(s_1, a_2)$
- $Q(s_1, a_3)$
- $Q(s_1, a_4)$

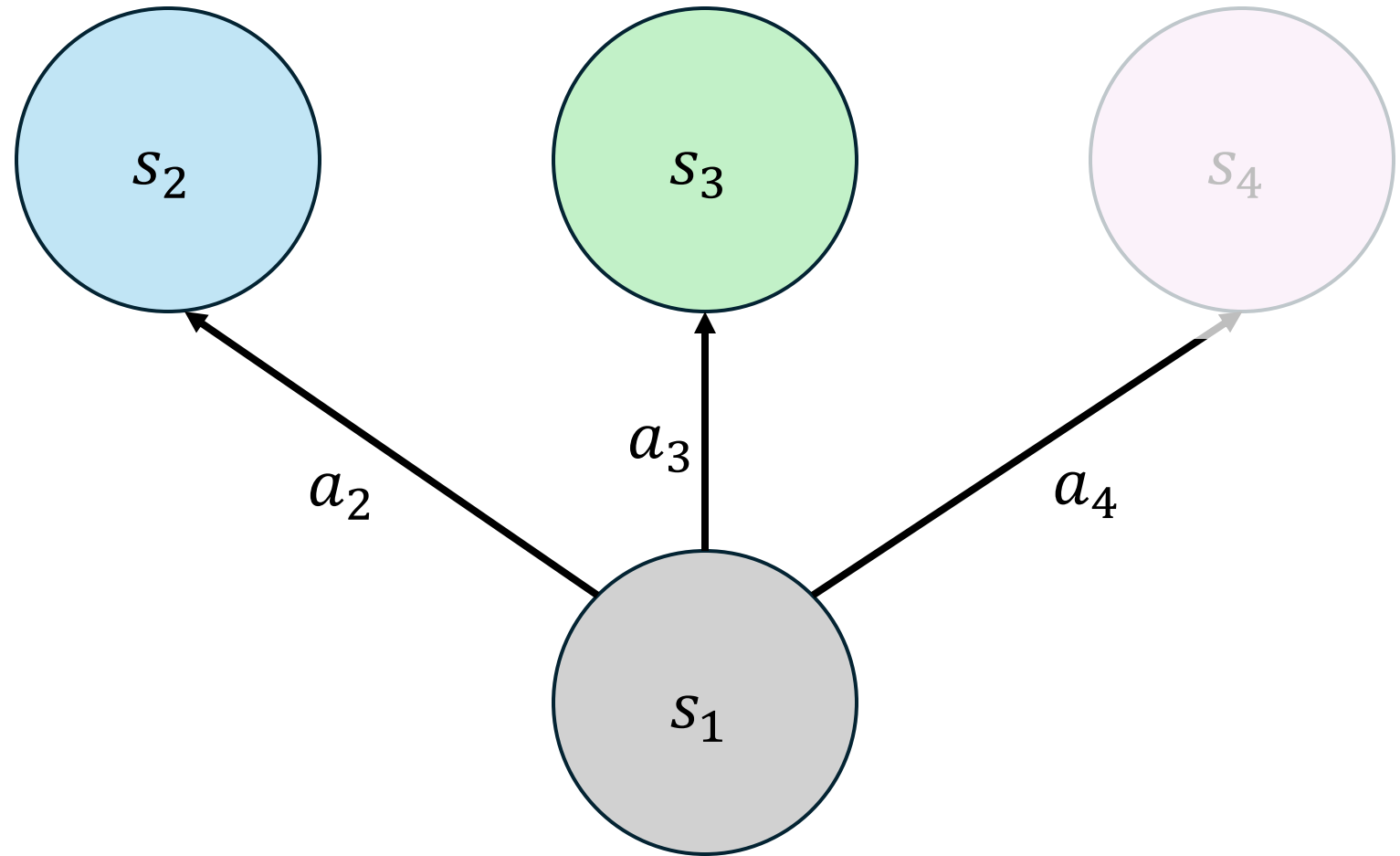


Extrapolation error

We want to estimate...

- $Q(s_1, a_2) = 1.2$
- $Q(s_1, a_3) = 2.8$
- $Q(s_1, a_4) = ???$

But our offline dataset doesn't contain (s_1, a_4)

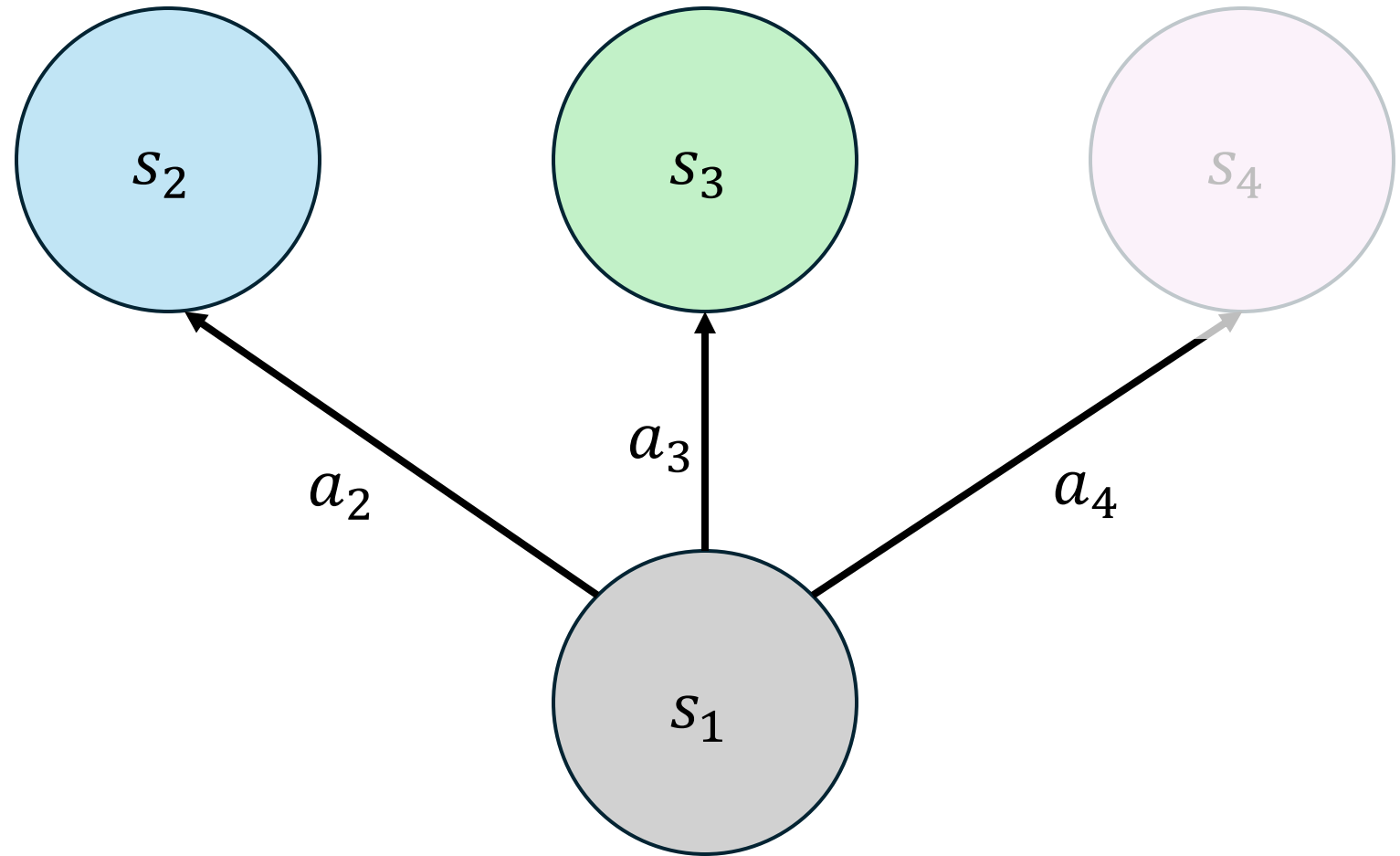


Extrapolation error

We want to estimate...

- $Q(s_1, a_2) = 1.2$
- $Q(s_1, a_3) = 2.8$
- $Q(s_1, a_4) = 3.3$

Suppose we predict an inaccurate value, we may blindly take this action!



Dealing with extrapolation error

Two types of offline reinforcement learning algorithms

Reward-weighted behavior cloning

- Doesn't estimate values at all, so avoids the issue

Conservative off-policy learning

- Avoid over-estimating values for state-action pairs not seen in the offline dataset

Reward-weighted behavior cloning

Standard behavior cloning loss augmented with an additional term

- Term proportional to reward

Higher reward: higher likelihood of taking this action in the learned policy

Lower reward: lower likelihood

$$\mathbb{E}_{s \sim d_{\pi}(s), a \sim \pi(a|s)} \exp(\beta \hat{A}^{\pi}(s, a)) \log \pi_{\theta}(a|s)$$

Conservative Q-learning

Learn a Q-function which lower-bounds the actual Q-function

- Goal: avoid over-estimating *any* value

Idea: minimize the expected Q-value for any given state while maximizing Q-values for states in the offline dataset

$$\hat{Q}_{\text{CQL}}^{\pi} := \arg \min_Q \alpha \cdot \left(\underbrace{\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})}[Q(\mathbf{s}, \mathbf{a})]}_{\text{minimize Q-values}} - \underbrace{\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \hat{\pi}_{\beta}(\mathbf{a}|\mathbf{s})}[Q(\mathbf{s}, \mathbf{a})]}_{\text{maximize Q-values under data}} \right) + \underbrace{\frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left(Q - \hat{B}^{\pi} Q \right)^2 \right]}_{\text{standard Bellman error}}$$

Kumar et al, Conservative Q-Learning for Offline Reinforcement Learning, 2020.

Conservative Q-learning

$$\hat{Q}_{\text{CQL}}^\pi := \arg \min_Q \alpha \cdot \left(\underbrace{\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})]}_{\text{minimize Q-values}} - \underbrace{\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \hat{\pi}_\beta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})]}_{\text{maximize Q-values under data}} \right) + \underbrace{\frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left(Q - \hat{B}^\pi Q \right)^2 \right]}_{\text{standard Bellman error}}$$

Minimize Q-values according to distribution μ

Maximize Q-values for state-action tuples in the training batch

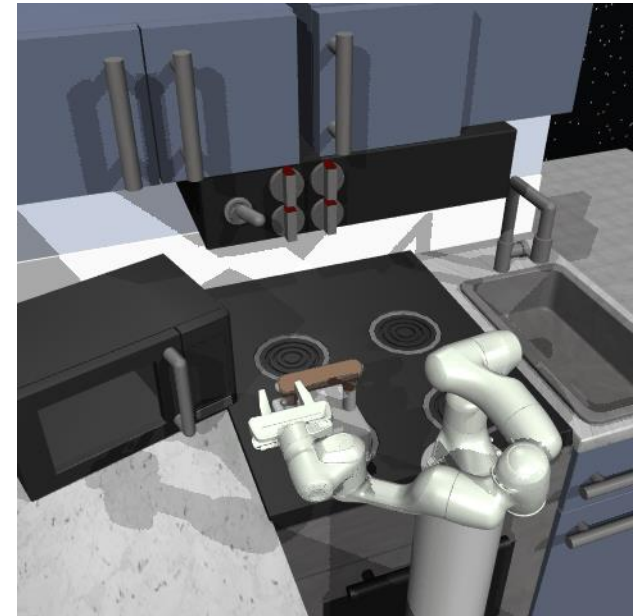
$$\min_Q \alpha \mathbb{E}_{\mathbf{s} \sim \mathcal{D}} \left[\log \sum_{\mathbf{a}} \exp(Q(\mathbf{s}, \mathbf{a})) - \mathbb{E}_{\mathbf{a} \sim \hat{\pi}_\beta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \right] + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left(Q - \hat{B}^{\pi_k} \hat{Q}^k \right)^2 \right].$$

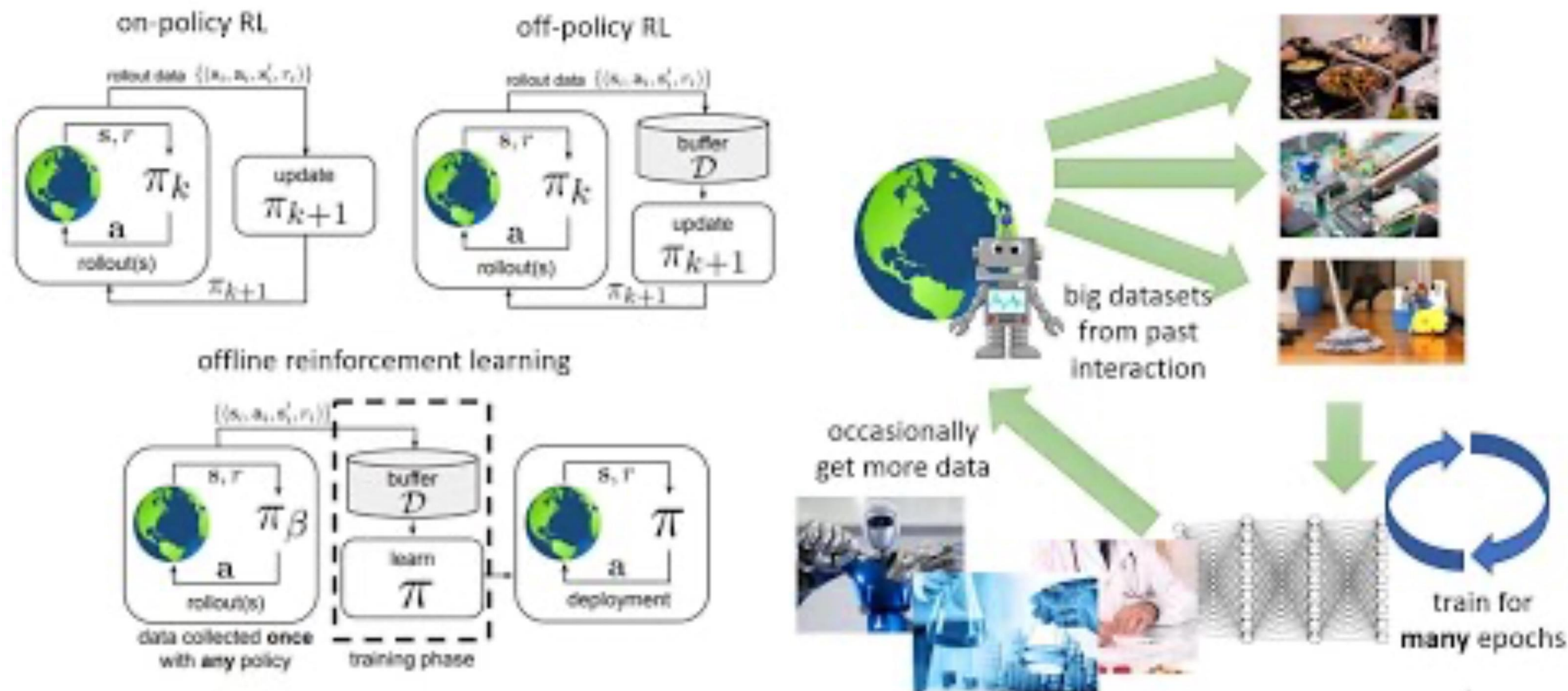
For example, a log-sum-exp which is dominated by the action with the largest Q-value, and hence minimizes the largest Q-value in each state

Algorithm 1 Conservative Q-Learning (both variants)

- 1: Initialize Q-function, Q_θ , and optionally a policy, π_ϕ .
 - 2: **for** step t in $\{1, \dots, N\}$ **do**
 - 3: Train the Q-function using G_Q gradient steps on objective from Equation 4
 $\theta_t := \theta_{t-1} - \eta_Q \nabla_\theta \text{CQL}(\mathcal{R})(\theta)$
 (Use $\mathcal{B}^*$ for Q-learning, $\mathcal{B}^{\pi_{\phi_t}}$ for actor-critic)
 - 4: (only with actor-critic) Improve policy π_ϕ via G_π gradient steps on ϕ with SAC-style entropy regularization:
 $\phi_t := \phi_{t-1} + \eta_\pi \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_\phi(\cdot|\mathbf{s})} [Q_\theta(\mathbf{s}, \mathbf{a}) - \log \pi_\phi(\mathbf{a}|\mathbf{s})]$
 - 5: **end for**
-

Domain	Task Name	BC	SAC	BEAR	BRAC-p	BRAC-v	CQL($\mathcal{H}$)	CQL(ρ)
AntMaze	antmaze-umaze	65.0	0.0	73.0	50.0	70.0	74.0	73.5
	antmaze-umaze-diverse	55.0	0.0	61.0	40.0	70.0	84.0	61.0
	antmaze-medium-play	0.0	0.0	0.0	0.0	0.0	61.2	4.6
	antmaze-medium-diverse	0.0	0.0	8.0	0.0	0.0	53.7	5.1
	antmaze-large-play	0.0	0.0	0.0	0.0	0.0	15.8	3.2
	antmaze-large-diverse	0.0	0.0	0.0	0.0	0.0	14.9	2.3
Adroit	pen-human	34.4	6.3	-1.0	8.1	0.6	37.5	55.8
	hammer-human	1.5	0.5	0.3	0.3	0.2	4.4	2.1
	door-human	0.5	3.9	-0.3	-0.3	-0.3	9.9	9.1
	relocate-human	0.0	0.0	-0.3	-0.3	-0.3	0.20	0.35
	pen-cloned	56.9	23.5	26.5	1.6	-2.5	39.2	40.3
	hammer-cloned	0.8	0.2	0.3	0.3	0.3	2.1	5.7
	door-cloned	-0.1	0.0	-0.1	-0.1	-0.1	0.4	3.5
	relocate-cloned	-0.1	-0.2	-0.3	-0.3	-0.3	-0.1	-0.1
Kitchen	kitchen-complete	33.8	15.0	0.0	0.0	0.0	43.8	31.3
	kitchen-partial	33.8	0.0	13.1	0.0	0.0	49.8	50.1
	kitchen-undirected	47.5	2.5	47.2	0.0	0.0	51.0	52.4





Imitation learning + reinforcement learning

Improving the sample efficiency of RL

(Online) reinforcement learning is extremely **sample inefficient**

- Especially model-free learning!

Why? Because the student learns everything on its own!

- Exploration vs exploitation

Imitation learning is sample efficient!

- Pure exploitation, teacher has already “explored” for us

Can we combine IL and RL?

Imitation Learning

Pros:

- Sample efficient
- Learns outside environment

Cons:

- Student performance is limited by optimality of teacher

Reinforcement Learning

Pros:

- Can (eventually) learn to optimally solve any task

Cons:

- Sample inefficient
- Learns in environment

Improving exploration with imitation learning

Can combine in different ways!

- Imitation learning first, then reinforcement learning
- Imitation learning and reinforcement learning at same time

Two general benefits:

1. IL can be used to “jump-start” exploration
2. IL can be used to learn hard-to-discover behaviors

Offline pre-training then online fine-tuning

Idea: pre-train policy using behavior cloning, then fine-tune with RL

- Initial policy for RL is a reasonable “first guess” instead of random

Behavior cloning fails to learn complex behaviors with limited demonstrations

- RL required to reach optimal

Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations

Aravind Rajeswaran^{1*}, Vikash Kumar^{1,2*}, Abhishek Gupta³, Giulia Vezzani⁴,
John Schulman², Emanuel Todorov¹, Sergey Levine³

Abstract—Dexterous multi-fingered hands are extremely versatile and provide a generic way to perform a multitude of tasks in human-centric environments. However, effectively controlling them remains challenging due to their high dimensionality and large number of potential contacts. Deep reinforcement learning (DRL) provides a model-agnostic approach to control complex dynamical systems, but has not been shown to scale to high-dimensional dexterous manipulation. Furthermore, deployment of DRL on physical systems remains challenging due to sample inefficiency. Consequently, the success of DRL in robotics has thus far been limited to simpler manipulators and tasks. In this work, we show that model-free DRL can effectively scale up to complex manipulation tasks with a high-dimensional 24-DoF hand, and solve them from scratch in simulated experiments. Furthermore, with the use of a small number of human demonstrations, the sample complexity can be significantly reduced, which enables learning with sample sizes equivalent to a few hours of robot experience. The use of demonstrations result in policies that exhibit very natural movements and, surprisingly, are also substantially more robust. We demonstrate successful policies for object relocation, in-hand manipulation, tool use, and door opening, which are shown in the supplementary video.

I. INTRODUCTION

Multi-fingered dexterous manipulators are crucial for robots to function in human-centric environments, due to their versatility and potential to enable a large variety of contact-rich tasks, such as in-hand manipulation, complex grasping, and tool use. However, this versatility comes at the price of high dimensional observation and action spaces, complex and discontinuous contact patterns, and under-actuation during non-prehensile manipulation. This makes dexterous manipulation with multi-fingered hands a challenging problem.

Dexterous manipulation behaviors with multi-fingered hands have previously been obtained using model-based trajec-

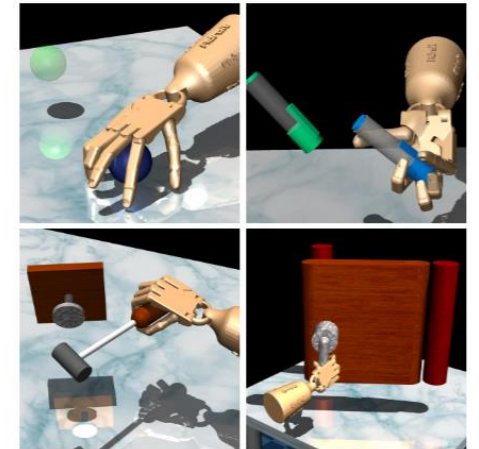


Figure 1: We demonstrate a wide range of dexterous manipulation skills such as object relocation, in-hand manipulation, tool use, and opening doors using DRL methods. By augmenting with human demonstrations, policies can be trained in the equivalent of a few real-world robot hours.

DRL research has made significant progress in improving performance on standardized benchmark tasks, such as the OpenAI gym benchmarks [6]. However, the current benchmarks are typically quite limited both in the dimensionality of the tasks and the complexity of the interactions. Indeed, recent work has shown that simple linear policies are capable of solving many of the widely studied benchmark tasks [40]. Thus, before we can develop DRL methods suitable for

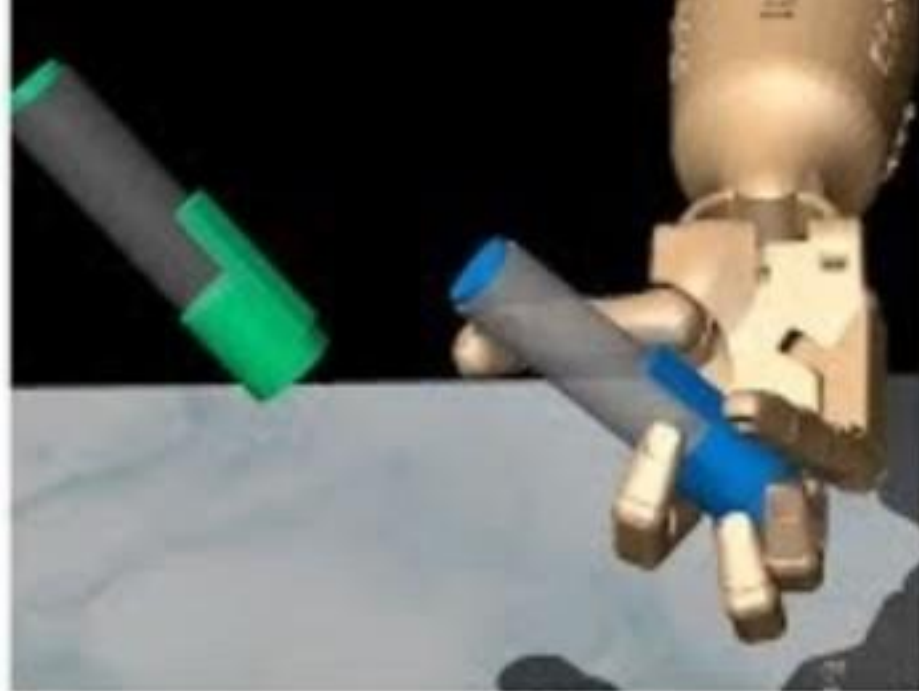
Offline pre-training then online fine-tuning

If we only use data for initialization, are we wasting it?

- What if demonstrations are of complex behavior, but require some baseline performance before they can be used?

Solution: add auxiliary loss to continue using data during RL

$$g_{aug} = \sum_{(s,a) \in \rho_{\pi}} \nabla_{\theta} \ln \pi_{\theta}(a|s) A^{\pi}(s, a) + \leftarrow \text{Update from policy experience}$$
$$\sum_{(s,a) \in \rho_D} \nabla_{\theta} \ln \pi_{\theta}(a|s) w(s, a). \leftarrow \text{Update from data}$$



How do we determine loss weighting?

Using offline data for online learning requires loss weighting

- How do we determine the weight? Tuned heuristic
- How do we know what data to use online?

Inefficient, requires lots of domain knowledge and tuning

Is there a better way?

Off-policy learning with demonstrations

Idea: learn initial policy and value function offline, then continue online

- Huge efficiency improvements even with small amount of data

Offline data is stored in replay buffer throughout entire training process

- When replay buffer is full, replace (oldest) online experiences
- No loss weighting/tuning required

Deep Q-Learning from Demonstrations

Todd Hester Google DeepMind toddhester@google.com	Matej Vecerik Google DeepMind matejvecerik@google.com	Olivier Pietquin Google DeepMind pietquin@google.com	Marc Lanctot Google DeepMind lanctot@google.com
Tom Schaul Google DeepMind schaul@google.com	Bilal Piot Google DeepMind piot@google.com	Dan Horgan Google DeepMind horgan@google.com	John Quan Google DeepMind johnquan@google.com
Andrew Sendonaris Google DeepMind sendos@yahoo.com	Ian Osband Google DeepMind iosband@google.com	Gabriel Dulac-Arnold Google DeepMind gabe@squirrelsoup.net	John Agapiou Google DeepMind jagapiou@google.com
Joel Z. Leibo Google DeepMind jzl@google.com		Audrunas Gruslys Google DeepMind audrunas@google.com	

Abstract

Deep reinforcement learning (RL) has achieved several high profile successes in difficult decision-making problems. However, these algorithms typically require a huge amount of data before they reach reasonable performance. In fact, their performance during learning can be extremely poor. This may be acceptable for a simulator, but it severely limits the applicability of deep RL to many real-world tasks, where the agent must learn in the real environment. In this paper we study a setting where the agent may access data from previous control of the system. We present an algorithm, *Deep Q-learning from Demonstrations* (DQfD), that leverages small sets of demonstration data to massively accelerate the learning process even from relatively small amounts of demonstration data and is able to automatically assess the necessary ratio of demonstration data while learning thanks to a prioritized replay mechanism. DQfD works by combining temporal difference updates with supervised classification of the demonstrator's actions. We show that DQfD has better initial performance than Prioritized Dueling Double Deep Q-Networks (PDD DQN) as it starts with better scores on the first million steps on 41 of 42 games and on average it takes PDD DQN 62 million steps to reach the same performance.

clude deep model-free Q-learning for general Atari game-playing ([Mnih et al. 2015](#)), end-to-end policy search for control of robot motors ([Levine et al. 2016](#)), model predictive control with embeddings ([Watter et al. 2015](#)), and strategic policies that combined with search led to defeating a top human expert at the game of Go ([Silver et al. 2016](#)). An important part of the success of these approaches has been to leverage the recent contributions to scalability and performance of deep learning ([LeCun, Bengio, and Hinton 2015](#)). The approach taken in ([Mnih et al. 2015](#)) builds a data set of previous experience using batch RL to train large convolutional neural networks in a supervised fashion from this data. By sampling from this data set rather than from current experience, the correlation in values from state distribution bias is mitigated, leading to good (in many cases, super-human) control policies.

It still remains difficult to apply these algorithms to real world settings such as data centers, autonomous vehicles ([Hester and Stone 2013](#)), helicopters ([Abbeel et al. 2007](#)), or recommendation systems ([Shani, Heckerman, and Brafman 2005](#)). Typically these algorithms learn good con-

Off-policy learning with demonstrations

Algorithm 1 Deep Q-learning from Demonstrations.

```
1: Inputs:  $\mathcal{D}^{replay}$ : initialized with demonstration data set,  
    $\theta$ : weights for initial behavior network (random),  $\theta'$ :  
   weights for target network (random),  $\tau$ : frequency at  
   which to update target net,  $k$ : number of pre-training  
   gradient updates  
2: for steps  $t \in \{1, 2, \dots, k\}$  do  
3:   Sample a mini-batch of  $n$  transitions from  $\mathcal{D}^{replay}$   
   with prioritization  
4:   Calculate loss  $J(Q)$  using target network  
5:   Perform a gradient descent step to update  $\theta$   
6:   if  $t \bmod \tau = 0$  then  $\theta' \leftarrow \theta$  end if  
7: end for  
8: for steps  $t \in \{1, 2, \dots\}$  do  
9:   Sample action from behavior policy  $a \sim \pi^{\epsilon Q_\theta}$   
10:  Play action  $a$  and observe  $(s', r)$ .  
11:  Store  $(s, a, r, s')$  into  $\mathcal{D}^{replay}$ , overwriting oldest  
   self-generated transition if over capacity  
12:  Sample a mini-batch of  $n$  transitions from  $\mathcal{D}^{replay}$   
   with prioritization  
13:  Calculate loss  $J(Q)$  using target network  
14:  Perform a gradient descent step to update  $\theta$   
15:  if  $t \bmod \tau = 0$  then  $\theta' \leftarrow \theta$  end if  
16:   $s \leftarrow s'$   
17: end for
```

Offline learning

Online learning

Dealing with extrapolation error

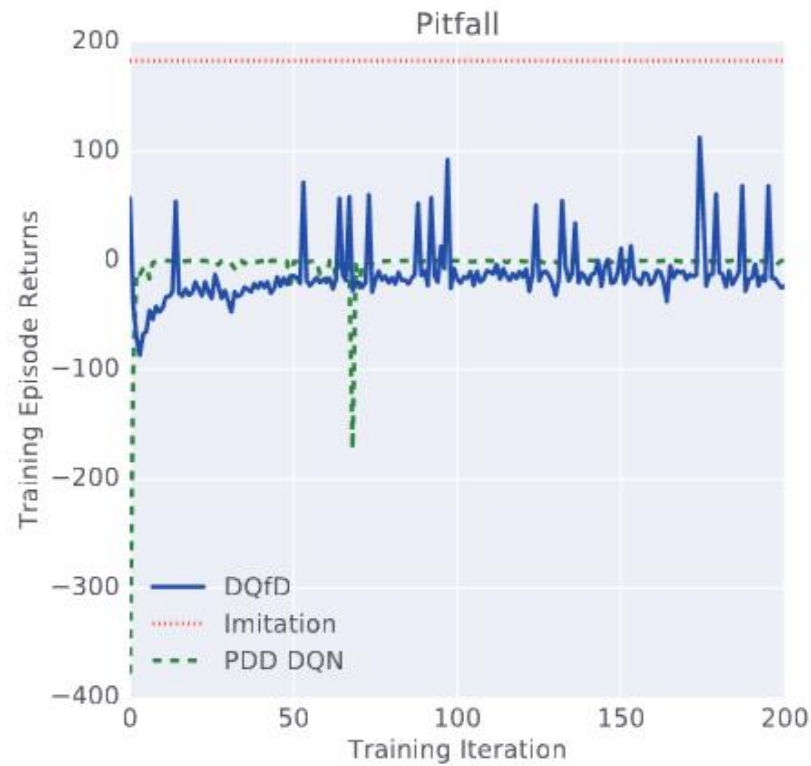
$$J_E(Q) = \max_{a \in A} [Q(s, a) + l(a_E, a)] - Q(s, a_E)$$


Large margin classification loss. 0 when $a_E = a$ and positive otherwise

Forces the the values of actions that the expert did not take to lower values. Grounds the values of unseen actions to “reasonable” values, and makes the greedy policy imitate the expert.

Other loss terms include n-step TD return and L2 regularization.

Off-policy learning with demonstrations



Side note: prioritized experience replay

Idea: uniformly sampling data from replay buffers is inefficient

- Some experience is helpful early
- Some is only helpful after agent has competency

Stochastically sample experience according to the last TD error magnitude

- Sample more frequently if a state has a higher expected learning progress

PRIORITIZED EXPERIENCE REPLAY

Tom Schaul, John Quan, Ioannis Antonoglou and David Silver

Google DeepMind

{schaul, johnquan, ioannisa, davidsilver}@google.com

ABSTRACT

Experience replay lets online reinforcement learning agents remember and reuse experiences from the past. In prior work, experience transitions were uniformly sampled from a replay memory. However, this approach simply replays transitions at the same frequency that they were originally experienced, regardless of their significance. In this paper we develop a framework for prioritizing experience, so as to replay important transitions more frequently, and therefore learn more efficiently. We use prioritized experience replay in Deep Q-Networks (DQN), a reinforcement learning algorithm that achieved human-level performance across many Atari games. DQN with prioritized experience replay achieves a new state-of-the-art, outperforming DQN with uniform replay on 42 out of 57 games.

1 INTRODUCTION

Online reinforcement learning (RL) agents incrementally update their parameters (of the policy, value function or model) while they observe a stream of experience. In their simplest form, they discard incoming data immediately, after a single update. Two issues with this are (a) strongly correlated updates that break the i.i.d. assumption of many popular stochastic gradient-based algorithms, and (b) the rapid forgetting of possibly rare experiences that would be useful later on.

Experience replay (Lin, 1992) addresses both of these issues: with experience stored in a replay memory, it becomes possible to break the temporal correlations by mixing more and less recent experience for the updates, and rare experience will be used for more than just a single update. This was demonstrated in the Deep Q-Network (DQN) algorithm (Mnih et al., 2013; 2015), which stabilized the training of a value function, represented by a deep neural network, by using experience replay. Specifically, DQN used a large sliding window replay memory, sampled from it uniformly at random, and effectively revisited each transition¹ eight times. In general, experience replay can reduce the amount of experience required to learn - and replace it with more computation and more

What about demonstration quality?

The previous works assume high-quality demonstration data

- Initially learn policy/value from this data
- Continue learning from it during online learning

But what if the demonstration data is low-quality?

Can we use offline data regardless of its optimality?

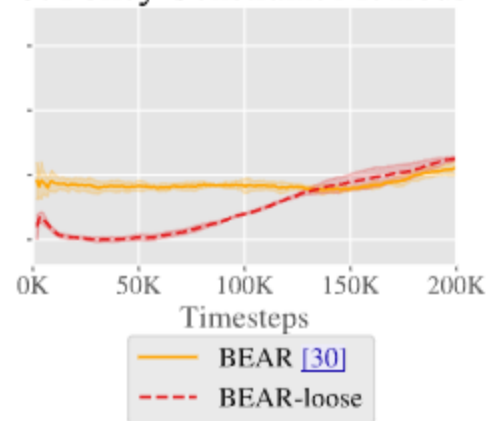
Can we simply run offline RL online?

Offline RL tackles demonstration quality issues

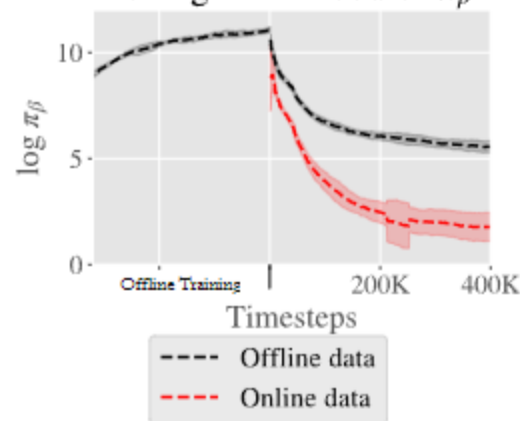
- Use rewards to weight state-actions so we only imitate the “best”

But applying online learning to policies learned offline doesn't work!

3. Policy Constraint Methods



4. Log Likelihood of π_β



Adapting offline RL to the online setting

When does offline RL fail in online?

- In “conservative” off-policy algos
- Why? Offline RL *discourages* exploring unseen actions
- Behavior model is often overfit and doesn’t incorporate new data well
- Exploration suffers and agent repeatedly performs sub-optimal actions

Solution: don’t model behavior policy

AWAC: Accelerating Online Reinforcement Learning with Offline Datasets

Ashvin Nair*, Abhishek Gupta*, Murtaza Dalal, Sergey Levine
Department of Electrical Engineering and Computer Science, UC Berkeley

Abstract—Reinforcement learning (RL) provides an appealing formalism for learning control policies from experience. However, the classic active formulation of RL necessitates a lengthy active exploration process for each behavior, making it difficult to apply in real-world settings such as robotic control. If we can instead allow RL algorithms to effectively use previously collected data to aid the online learning process, such applications could be made substantially more practical: the prior data would provide a starting point that mitigates challenges due to exploration and sample complexity, while the online training enables the agent to perfect the desired skill. Such prior data could either constitute expert demonstrations or, more generally, sub-optimal prior data that illustrates potentially useful transitions. While a number of prior methods have either used optimal demonstrations to bootstrap reinforcement learning, or have used sub-optimal data to train purely offline, it remains exceptionally difficult to train a policy with potentially sub-optimal offline data and actually continue to improve it further with online RL. In this paper we systematically analyze why this problem is so challenging, and propose an algorithm that combines sample-efficient dynamic programming with maximum likelihood policy updates, providing a simple and effective framework that is able to leverage large amounts of offline data and then quickly perform online fine-tuning of RL policies. We show that our method, advantage weighted actor critic (AWAC), enables rapid learning of skills with a combination of prior demonstration data and online experience. We demonstrate these benefits on a variety of simulated and real-world robotics domains, including dexterous manipulation with a real multi-fingered hand, drawer opening with a robotic arm, and rotating a valve. Our results show that incorporating prior data can reduce the time required to learn a range of robotic skills to practical time-scales.

I. INTRODUCTION



Figure 1: Utilizing prior data for online learning allows us to solve challenging real-world robotics tasks, such as this dexterous manipulation task where the learned policy must control a 4-fingered hand to reposition an object.

data effectively into the learning process, while still collecting additional data online for the task at hand. Doing so effectively will make the online data collection process much more practical while still allowing robots operating in the real world to continue improving their behavior.

For data-driven reinforcement learning, offline datasets consist of trajectories of states, actions and associated rewards. This data can potentially come from demonstrations for the desired task [48, 5], suboptimal policies [15], demonstrations for related tasks [63], or even just random exploration in the environment. Depending on the quality of the data that is provided, useful knowledge can be extracted about the dynamics of the world, about the task being solved, or both. Effective data-driven methods for deep reinforcement learning should be able to use this data to pre-train offline while improving with online fine-tuning.

Advantage Weighted Actor Critic

Essentially reward-weighted actor-critic

- Similar to reward-weighted behavior cloning, but this learns a value function

Does not suppress exploration but **prioritizes successful actions**

Works well in online learning!

Algorithm 1 Advantage Weighted Actor Critic (AWAC)

```
1: Dataset  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)_j\}$ 
2: Initialize buffer  $\beta = \mathcal{D}$ 
3: Initialize  $\pi_\theta, Q_\phi$ 
4: for iteration  $i = 1, 2, \dots$  do
5:   Sample batch  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r) \sim \beta$ 
6:    $y = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}', \mathbf{a}'} [Q_{\phi_{k-1}}(\mathbf{s}', \mathbf{a}')]$ 
7:    $\phi \leftarrow \arg \min_{\phi} \mathbb{E}_{\mathcal{D}} [(Q_{\phi}(\mathbf{s}, \mathbf{a}) - y)^2]$ 
8:    $\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \beta} [\log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \exp(\frac{1}{\lambda} A^{\pi_k}(\mathbf{s}, \mathbf{a}))]$ 
9:   if  $i > \text{num\_offline\_steps}$  then
10:     $\tau_1, \dots, \tau_K \sim p_{\pi_{\theta}}(\tau)$ 
11:     $\beta \leftarrow \beta \cup \{\tau_1, \dots, \tau_K\}$ 
12:   end if
13: end for
```

Advantage Weighted Actor Critic

